

Lab-7

Topic: BST, AVL trees

Q1: BST

Q2: Implementation and use of AVL trees

Q3: Implementation of Newick format

Q4: Insertion and Deletion of AVL trees (**Bonus**)

Q1 (30%)

Q1) BST from pre-order traversal.

You are provided with a sequence of integers representing the pre-order traversal of a BST.

Your task is to:

1. Construct the BST from the given Pre-Order traversal.
2. Once the BST is constructed, output the Post-Order traversal of the tree.

Input Format:

- Number of integers in sequence
- A sequence of integers (pre-order traversal) provided as input. All integers are distinct and represent nodes of a valid BST.

Output Format:

- The Post-Order traversal of the constructed BST.

Constraints:

- number of nodes ≤ 1000 .
- value of nodes ≤ 10000 .

Expected Complexity:

- Expected: $O(n)$ running time with $O(1)$ extra space for construction of BST, and $O(n)$ extra space for post-order traversal printing.
- You can also provide implementations that are of different time complexity (like $O(n^2)$), but such solutions will only fetch you partial marks (i.e. for smaller test cases). Solutions with more than constant extra space for construction can also be provided, and again they will fetch only partial marks.

Examples:

Input:

6

10 5 1 7 40 50

Output:

1 7 5 50 40 10

Q2 (40%)

Q2) You are tasked with implementing an AVL Tree (a height-balanced BST) as a C++ class.

Your task is to maintain the AVL balance at each node during insertion. The Operation of insertion is as follows:

`Insert(x)`: Insert a key x into the AVL. You need to implement the *iterative version*.

If it helps, you can start with an iterative implementation of BST insert, and then modify it to get an implementation of the AVL tree. If you need to store any additional information in a Tree Node (beyond data/key, left pointer, right pointer, and height fields), you are allowed to do so.

Input Format:

- Each operation will be an insertion Insert X where X is an integer value.
- End line will be PostOrder
- There might be insertion of duplicate elements into the AVL; these operations should be ignored without any error messages.

Output Format:

- Print one line representing POST ORDER traversal of the AVL.
- Number of operations $\leq 10^5$.
- Value of nodes $\leq 10^5$.

STL: You may use stack and/or vectors if needed.

Example:

Input1:

Insert 5
Insert 7
Insert 5
Insert 4
Insert 10
Insert 20
PostOrder

Output1:

4 7 20 10 5

Q3 (30%)

Q3)Write a program to parse a binary tree from Newick format, and a function to compute the height of this binary tree.

Newick format:

- The string $(A, B)C$ represents a binary tree where C is the root, A is the left child, and B is the right child.
- The string $(A)C$ represents a binary tree where C is the root, A is the left child, and B is the null.

You can use the Binary Tree class you have developed in the previous labs. Implement the following methods

- `BinaryTree& NewickToBinaryTree(string)`
- `bool IsBST(String) :` Should return true if the string representing a Binary tree in Newick format is a BST or not. One should not be converting the string to Binary Tree object.

Input format:

A binary tree in the Newick format.

Output format:

Inorder and preorder traversals of the binary tree.

Example:

Input:

((A,B)C,(D)F)G

Output:

A C B G D F

G C A B F D

Q4 Bonus (5 marks, capped to 5% globally)

Q4) Implement the delete operation on AVL trees that you have developed in Q2.

To question 1 above, add a non-recursive (iterative) implementation of deletion operation in the AVL Tree. So your extended code should support both these operations:

`Insert(x)` : Insert a key x into the AVL.

`Delete(x)` : Delete the node with key x from the AVL tree. Use the same logic as seen in class, including use of in-order successor to replace the deleted element if the node containing x has two (non-null) children.

Input Format:

- Each operation will either be an insertion `Insert X` or a deletion `Delete X`, where X is an integer value.
- End line will be `PostOrder`
- There might be insertion of duplicate elements to or removal of non-existent elements from the AVL; these operations should be ignored without any error messages.

Output Format:

- Print one line representing POST ORDER traversal of the AVL.

Post order traversal should also be implemented iteratively (using `std::stack` if needed).

Constraints:

- Number of operations $\leq 10^5$.
- Value of nodes $\leq 10^5$.

Example:**Input1:**

Insert 5

Insert 7

Insert 4

Insert 10

Insert 20

Delete 10

Insert 8

Delete 7

PostOrder

Output1:

4 20 8 5